PRAXIS PROGRAMMING

Game Project Report

EVIL LEARY

B37VB

Andrew Lima-Napier

al4038

Heriot-Watt University

Contents

Contents 2

Introduction 3

Project Aim 3

Program Structure 3

- **State-based flow** 3
- **Startup and asset loading** 4
- **Update and draw separation** 4

Main Systems 4

- **Movement and collision** 4
- **Dialogue and cutscenes** 4
- **Enemy behaviour** 4
- **Attack system** 5

Scene Progression 5

Final Fight Design 5

Controls and Rules 6

Testing and Iteration 6

Evaluation 7

- **What worked well** 7
- **What could be improved** 7

Conclusion 7

References 7

Introduction

EVIL LEARY is a 2D platform game that I built in C using raylib for the Praxis Programming module. I wanted the project to feel like a full game rather than just a small technical demo, so the final version includes a start menu, cutscenes, dialogue, chase sections, arena transitions, and a boss fight with health, hazards, and a visible timer.

The game follows a simple idea: the player is being chased by Evil Leary, escapes through a set of stages, and eventually reaches a final confrontation. Even though most of the project sits inside one main source file, the game still depends on a clear internal structure built around state changes, helper functions, and separate update and draw logic.

Project Aim

My aim was to make something that had pacing and personality instead of one mechanic repeated on one screen. I wanted the game to change mood as it went on, with slower sections for setup, faster sections for pressure, and a final phase that felt different enough to work as a proper ending.

From a programming point of view, the project was also meant to show that I could manage several systems at once. The final build uses input handling, animation, collision, scene transitions, dialogue flow, enemy behaviour, timers, and projectiles in one playable submission.

Program Structure

State-based flow

The most important design choice in the code is the use of game states. Instead of updating everything at all times, the program checks which part of the game is active and only runs the logic for that section. The menu only reacts to menu input, dialogue scenes pause gameplay, chase scenes run movement and enemy logic, and the boss fight enables its own hazards and timing rules. That approach made the project much easier to extend.

Startup and asset loading

When the game starts, it opens the raylib window, sets the frame rate, loads textures, generates tile data, and places the player and Kieran in the correct starting positions. The asset loading step matters because the game uses many sprites for tiles, props, character animations, interface elements, and special scenes. Once those files are loaded, the game can reuse them every frame without extra file overhead.

Update and draw separation

I also kept a clear split between updating the game and drawing the game. Update functions handle movement, collisions, timers, AI, dialogue progression, and transitions. Draw functions handle the background, platforms, characters, text boxes, health bars, and other visual layers. That split made debugging much more manageable, especially once the final fight became more complicated.

Main Systems

Movement and collision

Both the player and Kieran use the same basic actor structure. Each actor stores position, velocity, size, movement speed, jump strength, and direction. Gravity is applied every frame, then the collision system resolves whether the actor has landed on the ground or on one of the active platforms in the current scene. Reusing the same movement base for both characters kept the code simpler and made balancing much easier.

Dialogue and cutscenes

Dialogue is tied directly to the game state rather than being treated as decoration on top of gameplay. When dialogue is active, movement stops and the scene waits for the player to advance it. I used this in the opening cutscene, in the first arena, and again during the boss setup. This made the pacing feel more controlled and gave the story sections a clearer purpose.

Enemy behaviour

Kieran behaves differently depending on the stage. Early in the game he works as a chase enemy who follows the player and jumps when needed. Later, in the final fight, he becomes a boss-style threat with dialogue states, health, special movement, and a platform-based hazard

phase. That shift helps the game feel less repetitive because the same character creates pressure in different ways.

Attack system

The player attacks by pressing E. This starts an animation and then launches a book projectile once the throw frame is reached. In the final fight, those books damage Kieran and freeze him briefly, which gives the player a short chance to recover or reposition. I kept the attack system simple on purpose because the real challenge of the last stage comes from timing and movement, so the attack needed to stay readable.

Scene Progression

The game is built as a sequence of scenes rather than one continuous map. It starts at the menu, moves into an opening cutscene, becomes an active chase, pauses for arena dialogue, and then shifts into a final fight. That structure gave me a way to change the pace of the game so that the player is not doing exactly the same thing for the entire experience.

Transitions are also part of that structure. Instead of snapping between areas instantly, the game uses circular transitions and staged dialogue to move from one scene to the next. These details are small, but they help the project feel more deliberate and more complete.

Final Fight Design

The final fight is the most complex part of the project. By the time the player reaches it, the basic controls are already familiar, so the game can start asking for more accurate timing. The fight uses several systems at once: dialogue, moving and falling platforms, a health bar for Evil Leary, projectile damage, temporary freezes, and a countdown timer.

What I wanted from this section was a different kind of pressure from the earlier chase scenes. In the chase sections, the main question is whether the player can keep moving and stay ahead. In the final fight, the player has to survive hazards while also landing attacks. That difference is what gives the ending its own identity.

Controls and Rules

The control scheme is intentionally small so that the focus stays on timing and movement rather than on learning a lot of different inputs. Enter starts the game and advances dialogue. Movement is handled with A and D or the arrow keys where appropriate, jumping is handled with Space, W, or Up, and E performs the attack.

1. Enter starts the game and advances dialogue.
2. D or Right Arrow moves the player to the right.
3. A or Left Arrow is used where left movement is needed, especially in the final fight.
4. Space, W, or Up Arrow makes the player jump.
5. E triggers the attack and throws a book once the animation reaches the release frame.

The main gameplay rules are also fairly clear. If the player is caught in dangerous sections, the game resets or ends depending on the stage. Falling in the boss phase causes a game over. During the final fight, books reduce Kieran's health, and the player has to win before the timer expires.

Testing and Iteration

Most of the testing for the project was practical and iterative. Whenever I added a new mechanic or stage, I ran through the full game again to make sure earlier parts still worked. That mattered a lot because the project grew inside one source file, so a small change in one area could easily affect another if I was not careful.

A large part of the later work was balancing rather than just adding features. Jump heights, platform spacing, enemy distance, attack cooldown, freeze length, and falling-platform behaviour all had to be adjusted several times before the game felt fair. That process showed me that game development is not only about getting systems to work, but also about tuning them until they feel right.

Evaluation

What worked well

The part I am happiest with is the overall progression. The game does not stay locked into one mechanic for too long, and the switch from escape gameplay to boss survival makes the ending feel more memorable. I am also pleased with how the dialogue and transitions help connect the different sections together instead of making the game feel like a list of disconnected levels.

What could be improved

The clearest weakness is still the size of the main source file. Even though the code is grouped into sections, it would be better in the long run to split it into separate modules for rendering, actor logic, dialogue, and boss systems. I would also improve the presentation further with stronger sound design, more polished player feedback, and a clearer victory ending.

Conclusion

EVIL LEARY became much more than a simple platforming exercise. It includes menus, cutscenes, movement, collision, enemy logic, transitions, projectile combat, and a final fight with timed hazards. More importantly, it shows that a larger interactive program in C can still be kept under control when it is organised around states and helper functions.

There is still room to improve the structure and polish, but as a Praxis Programming submission it demonstrates a complete playable game with clear progression and several connected systems working together. That balance between technical implementation and game feel is what I was aiming for throughout the project.

References

raylib. Available at: https://www.raylib.com

itch Available at: itch.io

Craftpix Available at: craftpix.net

Sprite and texture assets were taken from the project materials and the external sources already referenced in the repository files.